Al-Imam University

Computer Science Department

College of Computer and Information Sciences

Design and Implementation of a Compiler Front-End for the HealthScript Language

Phase 2

Students:

Name	ID
Nuha Aedh Alrubayyi	444005638
Dan Riyad Alhumrani	444006468

CONTENTS

1. Parser Objective

The objective of Phase 2 is to design and implement a syntax analyzer for the HealthScript language. The parser receives the token stream generated by the lexical analyzer in Phase 1 and checks whether the program follows the grammar rules of the language. In this phase, a Context-Free Grammar is designed, transformed into an LL(1) suitable form, and used to build a predictive parser. The parser should accept valid HealthScript programs and report meaningful syntax errors for invalid programs.

2. Original CFG Grammar

```
<Program> → start <StmtList> finish

<StmtList> → <Stmt> <StmtList> | ε

<Stmt> → <VarDecl>
        | <AssignStmt>
        | <PrintStmt>
        | <ReadStmt>
        | <IfStmt>
        | <WhileStmt>
        | <FuncDecl>
        | <FuncCall>

<VarDecl> → var <Type> id ;

<Type> → int | float | string | bool

<AssignStmt> → id = <Expr> ;

<PrintStmt> → print <Expr> ;

<ReadStmt> → read id ;
```

<IfStmt> → if <BoolExpr> then <StmtList> <ElsePart> finish

<ElsePart> → else <StmtList> | ε

<WhileStmt> → while <BoolExpr> do <StmtList> finish

<FuncDecl> → func <ReturnType> id (<ParamList>) <StmtList> finish

<ReturnType> → int | float | string | bool | void

<ParamList> → <Param> <ParamRest> | ε

<ParamRest> → , <Param> <ParamRest> | ε

<Param> → <Type> id

<FuncCall> → do id (<ArgList>) ;

<ArgList> → <Expr> <ArgRest> | ε

<ArgRest> → , <Expr> <ArgRest> | ε

<BoolExpr> → <Expr> <RelOp> <Expr>

| <BoolExpr> or <BoolTerm>

| <BoolTerm>

<BoolTerm> → <BoolTerm> and <BoolFactor>

| <BoolFactor>

<BoolFactor> → not <BoolFactor>

| <Expr> <RelOp> <Expr>

| true

| false

<RelOp> → == | != | < | > | <= | >=

<Expr> → <Expr> + <Term>

| <Expr> - <Term>

```
        | <Term>
<Term> → <Term> * <Power>
        | <Term> / <Power>
        | <Term> % <Power>
        | <Power>
<Power> → <Factor> ^ <Power>
         | <Factor>
<Factor> → id
          | integer
          | float
          | string
          | true
          | false
          | ( <Expr> )
          | + <Factor>
          | - <Factor>
```

The original grammar describes the general structure of HealthScript programs. However, some expression rules contain left recursion, especially in arithmetic and Boolean expressions. Therefore, the grammar must be transformed to remove left recursion and become suitable for LL(1) predictive parsing.

3. LL(1)-Transformed Grammar

<Program> → start <StmtList> finish

<StmtList> → <Stmt> <StmtList> | ε

<Stmt> → <VarDecl>

| <AssignStmt>

| <PrintStmt>

| <ReadStmt>

| <IfStmt>

| <WhileStmt>

| <FuncDecl>

| <FuncCall>

<VarDecl> → var <Type> id ;

<Type> → int | float | string | bool

<AssignStmt> → id = <Expr> ;

<PrintStmt> → print <Expr> ;

<ReadStmt> → read id ;

<IfStmt> → if <BoolExpr> then <StmtList> <ElsePart> finish

<ElsePart> → else <StmtList> | ε

<WhileStmt> → while <BoolExpr> do <StmtList> finish

<FuncDecl> → func <ReturnType> id (<ParamList>) <StmtList> finish

<ReturnType> → int | float | string | bool | void

<ParamList> → <Param> <ParamRest> | ε

<ParamRest> → , <Param> <ParamRest> | ε

<Param> → <Type> id

<FuncCall> → do id (<ArgList>) ;

<ArgList> → <Expr> <ArgRest> | ε

<ArgRest> → , <Expr> <ArgRest> | ε

Expression Grammar with Precedence

<BoolExpr> → <BoolTerm> <BoolExpr'>

<BoolExpr'> → or <BoolTerm> <BoolExpr'> | ε

<BoolTerm> → <BoolFactor> <BoolTerm'>

<BoolTerm'> → and <BoolFactor> <BoolTerm'> | ε

<BoolFactor> → not <BoolFactor>

| <Expr> <RelPart>

| true

| false

<RelPart> → <RelOp> <Expr> | ε

<RelOp> → == | != | < | > | <= | >=

<Expr> → <Term> <Expr'>

<Expr'> → + <Term> <Expr'>

| - <Term> <Expr'>

| ε

<Term> → <Power> <Term'>

<Term'> → * <Power> <Term'>

| / <Power> <Term'>

| % <Power> <Term'>

| ε

```
<Power> → <Unary> <Power'>

<Power'> → ^ <Power> | ε

<Unary> → + <Unary>

        | - <Unary>

        | not <Unary>

        | <Factor>

<Factor> → id

         | integer

         | float

         | string

         | true

         | false

         | ( <Expr> )
```

The transformed grammar removes left recursion from arithmetic and Boolean expressions. It also separates expression levels to enforce operator precedence. The lowest precedence is handled by or, followed by and, relational operators, addition and subtraction, multiplication/division/modulo, power, and finally unary operators. The power operator is written as right-associative using the rule <Power'> → ^ <Power> | ε.

4. FIRST Sets

FIRST(<Program>) = { start }

FIRST(<StmtList>) = { var, id, print, read, if, while, func, do, ε }

FIRST(<Stmt>) = { var, id, print, read, if, while, func, do }

FIRST(<VarDecl>) = { var }

FIRST(<Type>) = { int, float, string, bool }

FIRST(<AssignStmt>) = { id }

FIRST(<PrintStmt>) = { print }

FIRST(<ReadStmt>) = { read }

FIRST(<IfStmt>) = { if }

FIRST(<ElsePart>) = { else, ε }

FIRST(<WhileStmt>) = { while }

FIRST(<FuncDecl>) = { func }

FIRST(<ReturnType>) = { int, float, string, bool, void }

FIRST(<ParamList>) = { int, float, string, bool, ε }

FIRST(<ParamRest>) = { ,, ε }

FIRST(<Param>) = { int, float, string, bool }

FIRST(<FuncCall>) = { do }

FIRST(<ArgList>) = { id, integer, float, string, true, false, (, +, -, not, ε }

FIRST(<ArgRest>) = { ,, ε }

FIRST(<BoolExpr>) = { id, integer, float, string, true, false, (, +, -, not }

FIRST(<BoolExpr'>) = { or, ε }

```
FIRST(<BoolTerm>) = { id, integer, float, string, true,
false, (, +, -, not }

FIRST(<BoolTerm'>) = { and, ε }

FIRST(<BoolFactor>) = { not, id, integer, float, string,
true, false, (, +, - }

FIRST(<RelPart>) = { ==, !=, <, >, <=, >=, ε }

FIRST(<RelOp>) = { ==, !=, <, >, <=, >= }

FIRST(<Expr>) = { id, integer, float, string, true,
false, (, +, -, not }

FIRST(<Expr'>) = { +, -, ε }

FIRST(<Term>) = { id, integer, float, string, true,
false, (, +, -, not }

FIRST(<Term'>) = { *, /, %, ε }

FIRST(<Power>) = { id, integer, float, string, true,
false, (, +, -, not }

FIRST(<Power'>) = { ^, ε }

FIRST(<Unary>) = { +, -, not, id, integer, float, string,
true, false, ( }

FIRST(<Factor>) = { id, integer, float, string, true,
false, ( }
```

5. FOLLOW Sets

```
FOLLOW(<Program>) = { $ }

FOLLOW(<StmtList>) = { finish, else, $ }

FOLLOW(<Stmt>) = { var, id, print, read, if, while, func,
do, finish, else, $ }

FOLLOW(<VarDecl>) = { var, id, print, read, if, while,
func, do, finish, else, $ }

FOLLOW(<Type>) = { id }
```

FOLLOW(<AssignStmt>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<PrintStmt>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<ReadStmt>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<IfStmt>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<ElsePart>) = { finish }

FOLLOW(<WhileStmt>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<FuncDecl>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<ReturnType>) = { id }

FOLLOW(<ParamList>) = {) }

FOLLOW(<ParamRest>) = {) }

FOLLOW(<Param>) = { ,,) }

FOLLOW(<FuncCall>) = { var, id, print, read, if, while, func, do, finish, else, $ }

FOLLOW(<ArgList>) = {) }

FOLLOW(<ArgRest>) = {) }

FOLLOW(<BoolExpr>) = { then, do,) }

FOLLOW(<BoolExpr'>) = { then, do,) }

FOLLOW(<BoolTerm>) = { or, then, do,) }

FOLLOW(<BoolTerm'>) = { or, then, do,) }

FOLLOW(<BoolFactor>) = { and, or, then, do,) }

FOLLOW(<RelPart>) = { and, or, then, do,) }

```
FOLLOW(<RelOp>) = { id, integer, float, string, true,
false, (, +, -, not }

FOLLOW(<Expr>) = { ;, ), ,, ==, !=, <, >, <=, >=, and,
or, then, do }

FOLLOW(<Expr'>) = { ;, ), ,, ==, !=, <, >, <=, >=, and,
or, then, do }

FOLLOW(<Term>) = { +, -, ;, ), ,, ==, !=, <, >, <=, >=,
and, or, then, do }

FOLLOW(<Term'>) = { +, -, ;, ), ,, ==, !=, <, >, <=, >=,
and, or, then, do }

FOLLOW(<Power>) = { *, /, %, +, -, ;, ), ,, ==, !=, <, >,
<=, >=, and, or, then, do }

FOLLOW(<Power'>) = { *, /, %, +, -, ;, ), ,, ==, !=, <,
>, <=, >=, and, or, then, do }

FOLLOW(<Unary>) = { ^, *, /, %, +, -, ;, ), ,, ==, !=, <,
>, <=, >=, and, or, then, do }

FOLLOW(<Factor>) = { ^, *, /, %, +, -, ;, ), ,, ==, !=,
<, >, <=, >=, and, or, then, do }
```

The FOLLOW sets represent the valid symbols that can appear immediately after each non-terminal during parsing. These sets are essential for constructing the LL(1) parsing table and ensuring that the parser can make correct predictive decisions without ambiguity.

6. LL(1) Parsing Table

The LL(1) parsing table is constructed using the FIRST and FOLLOW sets of the transformed grammar. Each table entry contains the production rule that should be applied when a specific non-terminal appears on the stack and a specific terminal appears in the input stream.The table allows the parser to make predictive parsing

decisions without backtracking, ensuring that the grammar satisfies the LL(1) property.

Table 1: Partial LL(1) Parsing Table

Non-Terminal	Input Symbol	Production Rule
<Program>	start	<Program> → start <StmtList> finish
<StmtList>	var	<StmtList> → <Stmt> <StmtList>
<StmtList>	id	<StmtList> → <Stmt> <StmtList>
<StmtList>	print	<StmtList> → <Stmt> <StmtList>
<StmtList>	read	<StmtList> → <Stmt> <StmtList>
<StmtList>	if	<StmtList> → <Stmt> <StmtList>
<StmtList>	while	<StmtList> → <Stmt> <StmtList>
<StmtList>	func	<StmtList> → <Stmt> <StmtList>
<StmtList>	do	<StmtList> → <Stmt> <StmtList>
<StmtList>	finish	<StmtList> → ε
<Stmt>	var	<Stmt> → <VarDecl>
<Stmt>	id	<Stmt> → <AssignStmt>
<Stmt>	print	<Stmt> → <PrintStmt>
<Stmt>	read	<Stmt> → <ReadStmt>
<Stmt>	if	<Stmt> → <IfStmt>
<Stmt>	while	<Stmt> → <WhileStmt>
<Stmt>	func	<Stmt> → <FuncDecl>

<Stmt>	do	<Stmt> → <FuncCall>
<VarDecl>	var	<VarDecl> → var <Type> id ;
<AssignStmt>	id	<AssignStmt> → id = <Expr> ;
<PrintStmt>	print	<PrintStmt> → print <Expr> ;
<ReadStmt>	read	<ReadStmt> → read id ;
<IfStmt>	if	<IfStmt> → if <BoolExpr> then <StmtList> <ElsePart> finish
<WhileStmt>	while	<WhileStmt> → while <BoolExpr> do <StmtList> finish
<FuncCall>	do	<FuncCall> → do id (<ArgList>) ;
<Expr>	id	<Expr> → <Term> <Expr'>
<Expr>	integer	<Expr> → <Term> <Expr'>
<Expr>	float	<Expr> → <Term> <Expr'>
<Expr>	string	<Expr> → <Term> <Expr'>
<Expr>	(	<Expr> → <Term> <Expr'>
<Expr'>	+	<Expr'> → + <Term> <Expr'>
<Expr'>	-	<Expr'> → - <Term> <Expr'>
<Expr'>	;	<Expr'> → ε
<Term'>	*	<Term'> → * <Power> <Term'>
<Term'>	/	<Term'> → / <Power> <Term'>
<Term'>	%	<Term'> → % <Power> <Term'>
<Term'>	+	<Term'> → ε
<Term'>	-	<Term'> → ε
<Term'>	;	<Term'> → ε

The parsing table demonstrates how predictive parsing decisions are made using one lookahead token. Each entry maps a non-terminal and an input symbol to the correct production rule. Empty entries in the table indicate syntax errors.

7. Parser Algorithm

The parser in HealthScript is implemented using the Recursive Descent Parsing technique, which is a top-down parsing approach suitable for LL(1) grammars. In this method, each non-terminal in the grammar is represented by a separate parsing function. The parser processes the token stream generated by the lexical analyzer and recursively applies grammar rules to verify the correctness of the program structure. The parsing process begins with the <Program> rule, which expects the program to start with the keyword start and end with finish. The parser then recursively processes statement lists, declarations, expressions, control structures, and function-related constructs according to the transformed LL(1) grammar. The parser uses a lookahead token mechanism to determine which production rule should be applied. At each step, the parser compares the current token with the expected token. If the token matches, the parser advances to the next token. Otherwise, a syntax error is reported.

The recursive descent parser includes parsing functions for:

- Program structure
- Statement lists
- Variable declarations
- Assignment statements
- Conditional statements
- Loop statements

- Function declarations and calls
- Arithmetic and Boolean expressions

Operator precedence is handled by separating expression parsing into multiple levels, including expressions, terms, powers, unary operations, and factors. This ensures correct parsing of arithmetic operations according to precedence and associativity rules. Error handling is also integrated into the parser. Whenever an unexpected token is encountered, the parser generates a descriptive syntax error message indicating the expected token, the received token, and the corresponding line number. The parser may also attempt simple recovery strategies to continue parsing the remaining input. Overall, the parser validates the syntactic correctness of HealthScript programs and provides structured error reporting, forming the second major phase of the compiler front-end.

8. Recursive Descent Parsing Procedure

1. Initialize the parser with the token stream generated by the scanner.
2. Set the current token as the first token in the stream.
3. Begin parsing using the Program() function.

4. For each non-terminal:

- Check the current token.
- Select the appropriate production rule.
- Match expected terminals.
- Recursively call parsing functions for non-terminals.

5. If the current token matches the expected token:

- Advance to the next token.

6. Otherwise:

- Generate a syntax error message.

7. Continue parsing until:

- The entire token stream is processed successfully, or
- A fatal syntax error is encountered.

8. If parsing completes successfully and the end-of-file symbol is reached:

- Accept the program as syntactically correct.

9. Parser Design Note

The parser was designed according to the LL(1) transformed grammar to ensure deterministic parsing without backtracking. Recursive descent parsing was selected because it is simple to implement, easy to debug, and well suited for educational compiler projects.

10. Parse Tree Representation

The parser generates a parse tree to represent the hierarchical syntactic structure of the input program according to the grammar rules of HealthScript. The parse tree visually demonstrates how tokens are derived from the grammar productions and how expressions and statements are organized within the program. Each internal node in the parse tree represents a non-terminal symbol, while leaf nodes represent terminal symbols (tokens). The parse tree helps verify that the parser correctly applies

production rules and respects operator precedence and associativity. The parse tree also assists in debugging and validating the correctness of the parser implementation.

Note

The parser generates and displays the parse tree dynamically during execution, as shown in the test case screenshots.

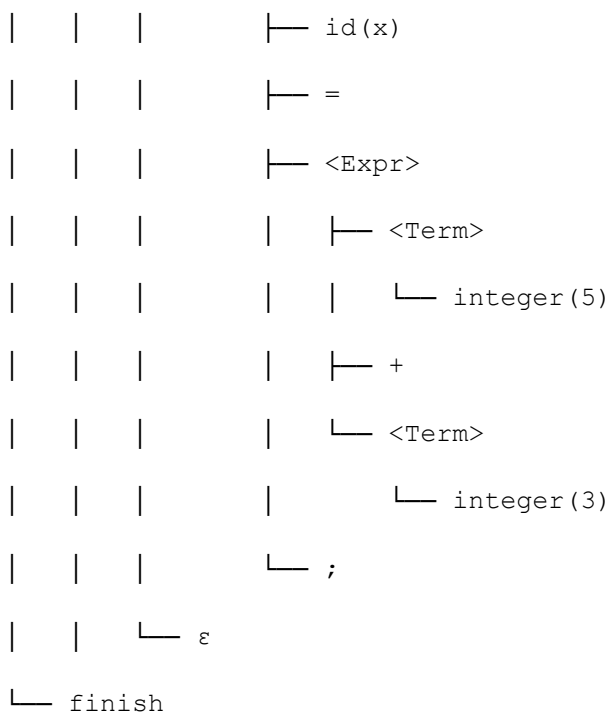
Example Parse Tree

Example Input Program:

```
start
var int x;
x = 5 + 3;
finish
```

(Consolas)

```
<Program>
├── start
├── <StmtList>
│   ├── <Stmt>
│   │   └── <VarDecl>
│   │       ├── var
│   │       ├── int
│   │       ├── id(x)
│   │       └── ;
│   ├── <StmtList>
│   │   ├── <Stmt>
│   │   │   └── <AssignStmt>
```



The parse tree above illustrates how the parser derives the input program according to the LL(1) grammar rules. It demonstrates the hierarchical relationship between declarations, assignments, and arithmetic expressions.

11. Syntax Error Handling

The HealthScript parser includes a syntax error handling mechanism to detect invalid program structures during parsing. Whenever the parser encounters an unexpected token that does not match the expected grammar rule, it generates a descriptive syntax error message.

The error message includes:

- The line number where the error occurred
- The unexpected token received
- The expected token or grammar construct

- Syntax error messages include the line number, the expected token, and the received token.

This helps users identify and correct syntax mistakes efficiently.

Examples of syntax errors detected by the parser include:

- Missing semicolons
- Missing finish keyword
- Invalid statement structure
- Unmatched parentheses
- Invalid expressions
- Incorrect function declarations
- Missing then or do keywords

The parser may also apply simple recovery techniques, such as skipping invalid tokens or continuing parsing from the next statement, in order to report multiple syntax errors within the same program.

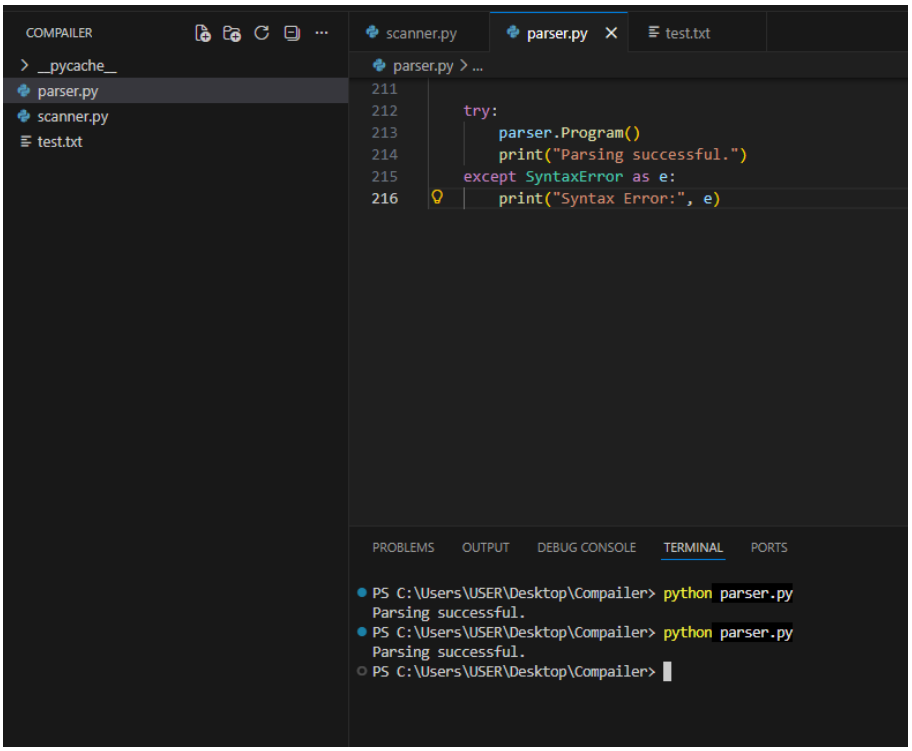
12. Parser Test Cases

Test Case 1: Valid Program

Code

```
start
var int x;
x = 5 + 3;
print x;
```

```
finish
```



Output

Parsing successful.

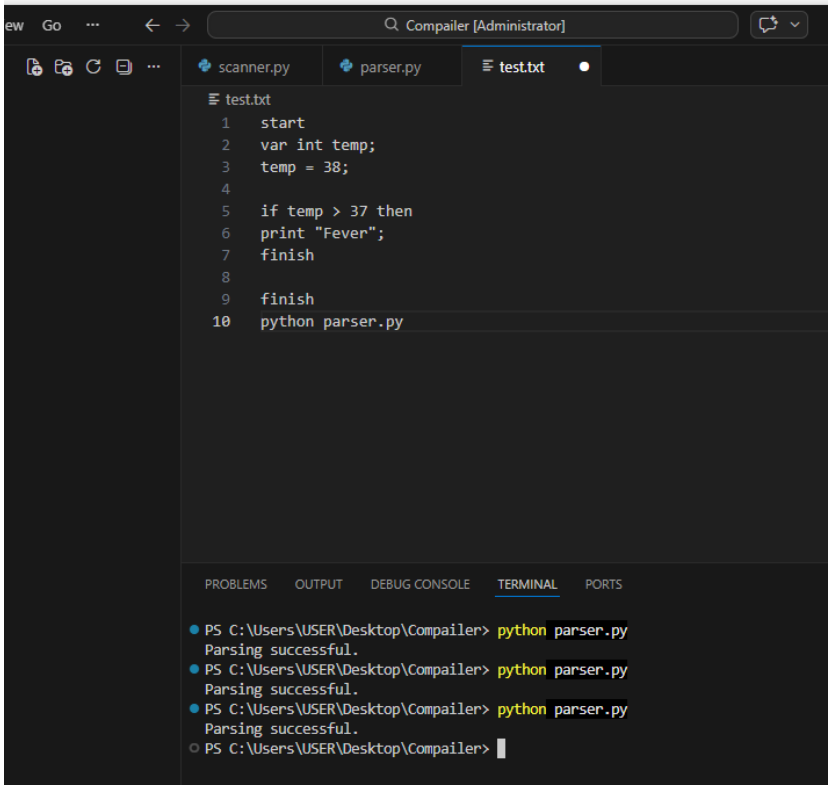
No syntax errors found.

Test Case 2: Valid Conditional Statement

Code

```
start
var int temp;
temp = 38;
f temp > 37 then
print "Fever";
finish
```

```
finish
```



Output

Parsing successful.

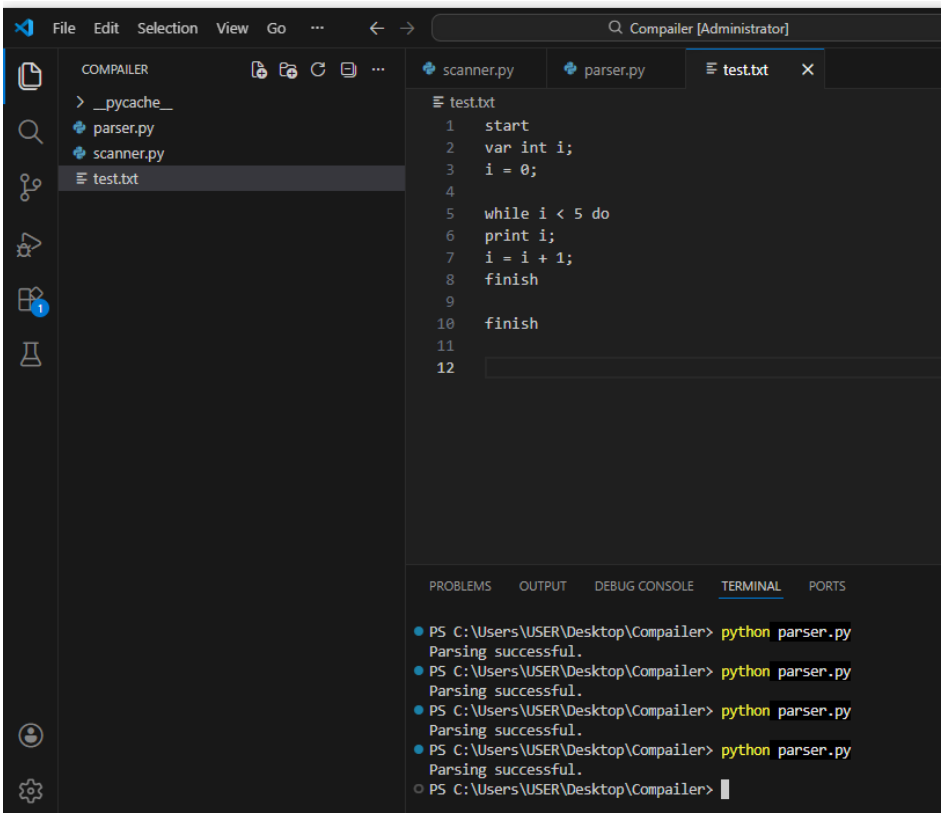
No syntax errors found.

Test Case 3: Valid Loop Statement

Code

```
start

var int i;

i = 0;

while i < 5 do

print i;
```

```
i = i + 1;

finish

finish
```



Output

Parsing successful.

No syntax errors found.

Test Case 4: Missing Semicolon

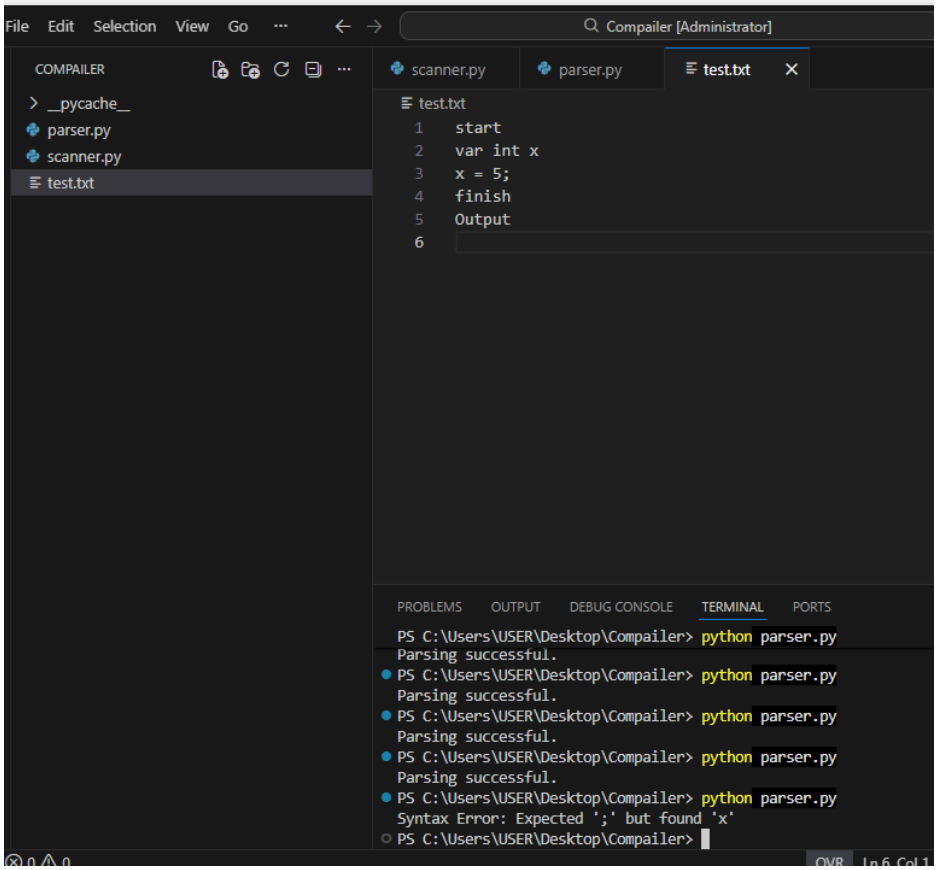
Code

start

var int x

x = 5;

finish



Output

Syntax Error: Expected ';' but found 'x'

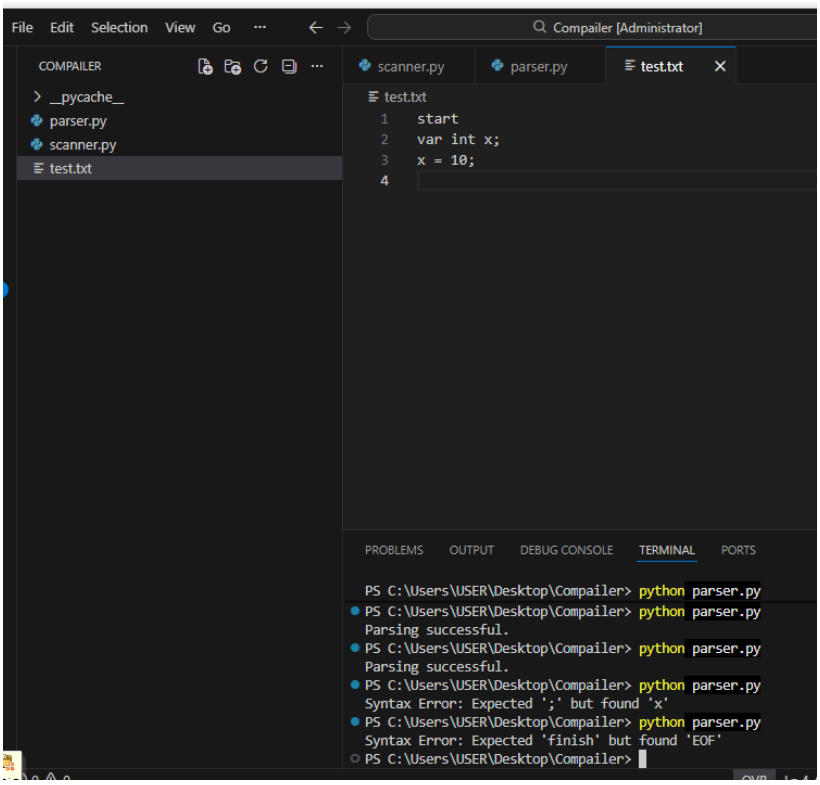
Test Case 5: Missing finish Keyword

Code

start

var int x;

x = 10;



Output

Syntax Error:

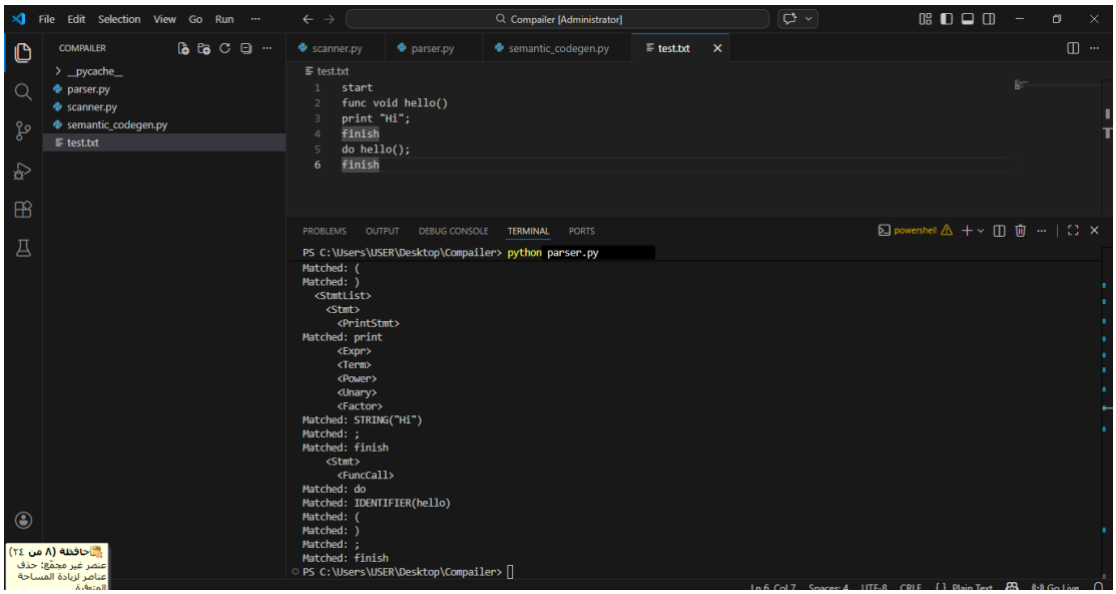
Expected 'finish' at end of program.

The test cases demonstrate the ability of the parser to correctly validate valid HealthScript programs and detect syntax errors in invalid programs. Both successful parsing and syntax error reporting were verified during testing.

Test Case 6 : Function Declaration and Function Call

Code

```
start

func void hello()

print "Hi";

finish

do hello();

finish
```



Output:

Parsing successful.

The parser successfully recognized the function declaration, the print statement inside the function body, and the function call statement. The parse tree was generated and displayed in the terminal, showing the matched tokens and grammar structure for the function declaration and function call.

Test Case 7 : for and / or / not

start

var int x;

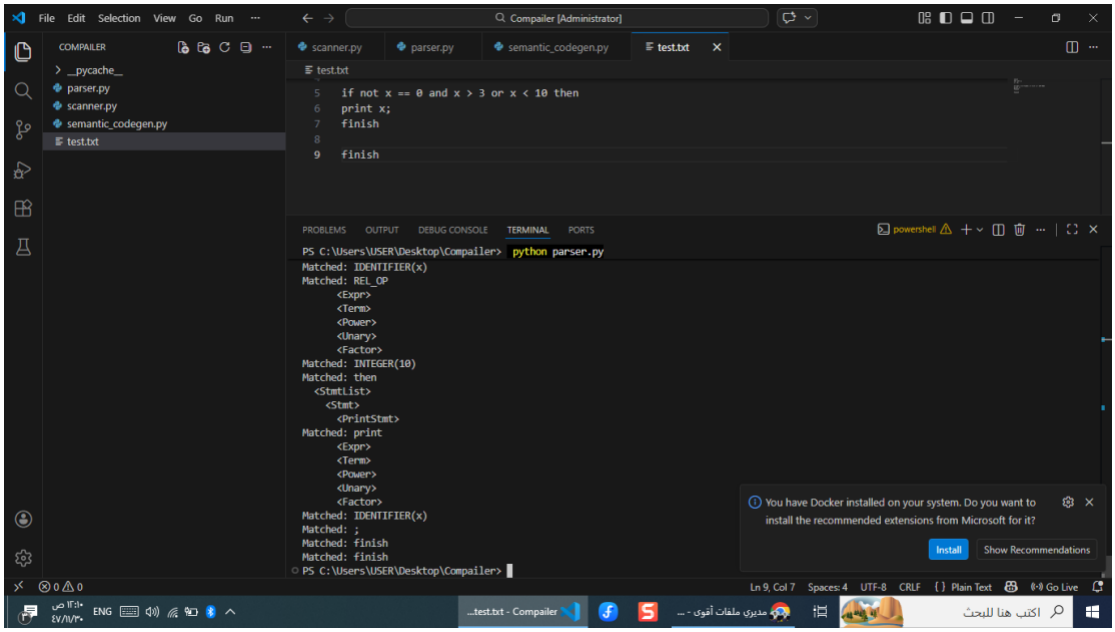
x = 5;

if not x == 0 and x > 3 or x < 10 then

print x;

finish

finish



output

Parsing successful.

Parse Tree Display

Case Study 8 : line number

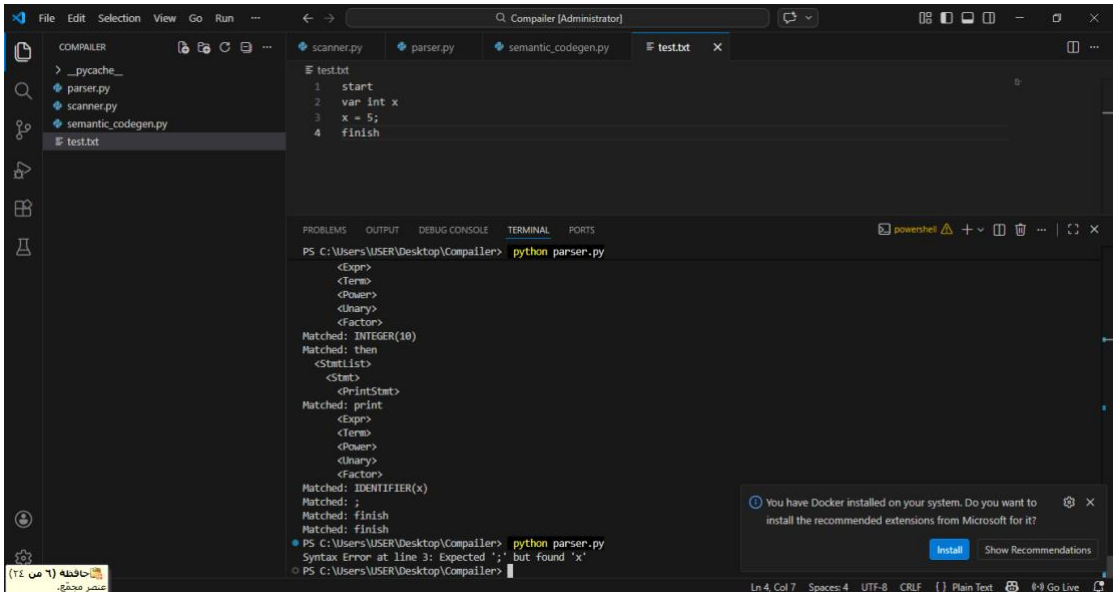
Code

start

var int x

x = 5;

finish



Output

Syntax Error at line ...

13. Conclusion

In this phase, a complete syntax analyzer for the HealthScript language was successfully designed and implemented. A Context-Free Grammar (CFG) was created to define the structure of the language, including declarations, assignments, expressions, control structures, functions, and input/output statements. The grammar was transformed into an LL(1)-compatible form by eliminating left recursion and applying grammar transformations suitable for predictive parsing. FIRST and FOLLOW sets were constructed, and a partial LL(1) parsing table was generated to support parsing decisions. A recursive descent parser was designed to validate token streams produced by the lexical analyzer from Phase 1. The parser correctly recognized syntactically valid programs and generated meaningful syntax error messages for invalid inputs. The implementation demonstrates the fundamental principles of syntax analysis and predictive parsing in compiler construction. This phase also establishes the foundation for future compiler stages such as semantic analysis and code generation.

14. Appendix: Parser Source Code

The following appendix contains the main implementation of the HealthScript recursive descent parser developed in Python. The parser processes token streams generated by the lexical analyzer and validates them according to the LL(1) grammar rules.

15. Parser Implementation (Python)

The parser was implemented using the Recursive Descent Parsing technique in Python. Each non-terminal in the grammar is represented as a parsing function. The

parser reads the token stream generated by the lexical analyzer and validates the syntax according to the LL(1) grammar rules.

Parser Source Code

```
from scanner import healthscript_scanner

class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.current = 0
        self.tree = []

    def current_token(self):
        if self.current < len(self.tokens):
            return self.tokens[self.current]
        return ("EOF", -1, "EOF")

    def add_tree(self, text):
        self.tree.append(text)

    def match(self, expected):
        token_type, line, value = self.current_token()
        if value == expected:
            self.add_tree(f"Matched: {value}")
            self.current += 1
        else:
            raise SyntaxError(
                f"Syntax Error at line {line}: Expected '{expected}'
but found '{value}'"
            )

    def match_type(self, expected_type):
        token_type, line, value = self.current_token()
        if token_type == expected_type:
            self.add_tree(f"Matched: {token_type}({value})")
            self.current += 1
        else:
            raise SyntaxError(
                f"Syntax Error at line {line}: Expected {expected_type}
but found '{value}'"
            )

    def Program(self):
        self.add_tree("<Program>")
```

```
        self.match("start")
        self.StmtList()
        self.match("finish")

    def StmtList(self):
        self.add_tree("  <StmtList>")
        while self.current_token()[2] in ["var", "print", "read", "if",
"while", "do", "func"] or self.current_token()[0] == "IDENTIFIER":
            self.Stmt()

    def Stmt(self):
        value = self.current_token()[2]
        token_type = self.current_token()[0]

        self.add_tree("    <Stmt>")

        if value == "var":
            self.VarDecl()
        elif token_type == "IDENTIFIER":
            self.AssignStmt()
        elif value == "print":
            self.PrintStmt()
        elif value == "read":
            self.ReadStmt()
        elif value == "if":
            self.IfStmt()
        elif value == "while":
            self.WhileStmt()
        elif value == "do":
            self.FuncCall()
        elif value == "func":
            self.FuncDecl()
        else:
            line = self.current_token()[1]
            raise SyntaxError(f"Syntax Error at line {line}: Unexpected
token '{value}'")

    def VarDecl(self):
        self.add_tree("      <VarDecl>")
        self.match("var")
        self.Type()
        self.match_type("IDENTIFIER")

        if self.current_token()[2] == "=":
            self.match("=")
            self.Expr()
```

```
        self.match(";")

    def Type(self):
        value = self.current_token()[2]
        line = self.current_token()[1]

        if value in ["int", "float", "string", "bool"]:
            self.add_tree(f"      <Type> {value}")
            self.current += 1
        else:
            raise SyntaxError(f"Syntax Error at line {line}: Expected
data type")

    def ReturnType(self):
        value = self.current_token()[2]
        line = self.current_token()[1]

        if value in ["int", "float", "string", "bool", "void"]:
            self.add_tree(f"      <ReturnType> {value}")
            self.current += 1
        else:
            raise SyntaxError(f"Syntax Error at line {line}: Expected
return type")

    def AssignStmt(self):
        self.add_tree("      <AssignStmt>")
        self.match_type("IDENTIFIER")
        self.match("=")
        self.Expr()
        self.match(";")

    def PrintStmt(self):
        self.add_tree("      <PrintStmt>")
        self.match("print")
        self.Expr()
        self.match(";")

    def ReadStmt(self):
        self.add_tree("      <ReadStmt>")
        self.match("read")
        self.match_type("IDENTIFIER")
        self.match(";")

    def IfStmt(self):
        self.add_tree("      <IfStmt>")
        self.match("if")
        self.BoolExpr()
```

```
        self.match("then")
        self.StmtList()

        if self.current_token()[2] == "else":
            self.match("else")
            self.StmtList()

        self.match("finish")

    def WhileStmt(self):
        self.add_tree("      <WhileStmt>")
        self.match("while")
        self.BoolExpr()
        self.match("do")
        self.StmtList()
        self.match("finish")

    def FuncDecl(self):
        self.add_tree("      <FuncDecl>")
        self.match("func")
        self.ReturnType()
        self.match_type("IDENTIFIER")
        self.match("(")

        if self.current_token()[2] != ")":
            self.ParamList()

        self.match(")")
        self.StmtList()
        self.match("finish")

    def ParamList(self):
        self.add_tree("      <ParamList>")
        self.Type()
        self.match_type("IDENTIFIER")

        while self.current_token()[2] == ",":
            self.match(",")
            self.Type()
            self.match_type("IDENTIFIER")

    def FuncCall(self):
        self.add_tree("      <FuncCall>")
        self.match("do")
        self.match_type("IDENTIFIER")
        self.match("(")
```

```
        if self.current_token()[2] != ")":
            self.ArgList()

        self.match(")")
        self.match(";")

    def ArgList(self):
        self.add_tree("      <ArgList>")
        self.Expr()

        while self.current_token()[2] == ",":
            self.match(",")
            self.Expr()

    def BoolExpr(self):
        self.add_tree("      <BoolExpr>")
        self.BoolTerm()

        while self.current_token()[2] == "or":
            self.match("or")
            self.BoolTerm()

    def BoolTerm(self):
        self.add_tree("      <BoolTerm>")
        self.BoolFactor()

        while self.current_token()[2] == "and":
            self.match("and")
            self.BoolFactor()

    def BoolFactor(self):
        self.add_tree("      <BoolFactor>")

        if self.current_token()[2] == "not":
            self.match("not")
            self.BoolFactor()
        else:
            self.Expr()

            if self.current_token()[2] in ["==", "!=", "<", ">", "<=",
">="]:
                self.current += 1
                self.add_tree("Matched: REL_OP")
                self.Expr()

    def Expr(self):
        self.add_tree("      <Expr>")
```

```
        self.Term()

        while self.current_token()[2] in ["+", "-"]:
            self.current += 1
            self.add_tree("Matched: ADD_OP")
            self.Term()

    def Term(self):
        self.add_tree("      <Term>")
        self.Power()

        while self.current_token()[2] in ["*", "/", "%"]:
            self.current += 1
            self.add_tree("Matched: MUL_OP")
            self.Power()

    def Power(self):
        self.add_tree("      <Power>")
        self.Unary()

        if self.current_token()[2] == "^":
            self.match("^")
            self.Power()

    def Unary(self):
        self.add_tree("      <Unary>")

        if self.current_token()[2] in ["+", "-", "not"]:
            op = self.current_token()[2]
            self.match(op)
            self.Unary()
        else:
            self.Factor()

    def Factor(self):
        token_type, line, value = self.current_token()
        self.add_tree("      <Factor>")

        if token_type in ["IDENTIFIER", "INTEGER", "FLOAT", "STRING"]:
            self.add_tree(f"Matched: {token_type}({value})")
            self.current += 1
        elif value in ["true", "false"]:
            self.add_tree(f"Matched: BOOL({value})")
            self.current += 1
        elif value == "(":
            self.match("(")
            self.Expr()
```

```
            self.match(")")
        else:
            raise SyntaxError(
                f"Syntax Error at line {line}: Unexpected token
'{value}' in expression"
            )

    def display_parse_tree(self):
        print("\n--- Parse Tree Display ---")
        for item in self.tree:
            print(item)


with open("test.txt", "r") as file:
    source_code = file.read()

total, tokens, sym_table, lexical_errors =
healthscript_scanner(source_code)

if lexical_errors:
    print("Lexical errors found:")
    for err in lexical_errors:
        print(err)
else:
    parser_tokens = [(t[0], t[1], t[2]) for t in tokens]
    parser = Parser(parser_tokens)

    try:
        parser.Program()
        print("Parsing successful.")
        parser.display_parse_tree()
    except SyntaxError as e:
        print(e)
```

The parser implementation demonstrates the core functionality of recursive descent parsing for the HealthScript language. The parser validates declarations, assignments, expressions, conditional statements, loops, and function calls according to the LL(1) grammar rules.